

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN



Proyecto final

PRESENTADO POR LOS ALUMNOS

[ENRIQUE ALONSO ACUÑA YEPEZ]

[LLUVIA DANIELA CASTILLO CASTILLO]

[RAMSÉS MIJAÍL RAFAEL VALENZUELA]

Código

```
# scraper_tijuana.py — Vivanuncios ETL Module
# Scrapes commercial rental listings from vivanuncios.com.mx
# for four Mexican cities (Tijuana, CDMX, Monterrey,
# Guadalajara), cleans the price data, and loads it into
# both CSV files and MySQL via stored procedures.
# We chose requests + BeautifulSoup over Selenium because
# Vivanuncios renders its listing cards in server-side HTML,
# so no JavaScript execution is needed. This makes the
# scraper faster and dependency-lighter.
#
# ETL flow:
# extract_city() → transform() → save_csv() + save_mysql()
# All orchestrated by scrape_vivanuncios() at the bottom.

import os
import re
import csv
import time
import random
import requests
import mysql.connector
from datetime import date
from bs4 import BeautifulSoup
from config import DATA_DIR, MYSQL_HOST, MYSQL_USER, MYSQL_PASS,
MYSQL_DB

# CITIES — Target URLs per city
# Each URL template accepts a {page} placeholder for
# pagination. The URL structure encodes both the listing
# category (renta-locales-comerciales) and the geographic
# area via Vivanuncios internal location codes (110015, etc.).
CITIES = {
    "Tijuana":
        "https://www.vivanuncios.com.mx/s-renta-locales-comerciales/tijuana/v1c1276l10015p{page}",
    "CDMX":
        "https://www.vivanuncios.com.mx/s-renta-locales-comerciales/ciudad-de-mexico/v1c1276l10486l10487p{page}",
    "Monterrey":
        "https://www.vivanuncios.com.mx/s-renta-locales-comerciales/nuevo-leon/monterrey/v1c1276l10379l10380p{page}",
    "Guadalajara":
        "https://www.vivanuncios.com.mx/s-renta-locales-comerciales/jalisco/guadalajara/v1c1276l10023l10024p{page}",
```

```
}
```

```
CSV_FIELDS = ["city", "neighborhood", "price", "currency", "scraped_date"] # --  
consistent column order for all CSV outputs
```

```
# HEADERS — HTTP request headers
```

```
# We mimic a real Chrome browser to avoid bot-detection
```

```
# by Vivanuncios's Cloudflare layer. The Referer header
```

```
# simulates navigation from the site's own homepage,
```

```
# which is the most common legitimate pattern.
```

```
HEADERS = {
```

```
    "User-Agent": (
```

```
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
```

```
        "AppleWebKit/537.36 (KHTML, like Gecko) "
```

```
        "Chrome/124.0.0.0 Safari/537.36" # -- mimics a real Chrome 124 browser to
```

```
bypass bot filters
```

```
    ),
```

```
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
```

```
    "Accept-Language": "es-MX,es;q=0.9", # -- Spanish-MX locale avoids geo-redirect  
issues
```

```
    "Referer": "https://www.vivanuncios.com.mx/", # -- simulates organic navigation  
from the homepage
```

```
}
```

```
def get_connection():
```

```
    return mysql.connector.connect(
```

```
        host = MYSQL_HOST,
```

```
        user = MYSQL_USER,
```

```
        password = MYSQL_PASS,
```

```
        database = MYSQL_DB,
```

```
        auth_plugin = "mysql_native_password" # -- forces legacy auth to avoid SHA2
```

```
handshake issues
```

```
    )
```

```
# parse_price_currency — Extracts price and currency from a raw string
```

```
# Vivanuncios listings mix USD and MXN prices in the same
```

```
# text field with inconsistent formatting. We try three
```

```
# regex patterns in priority order:
```

```
# 1. "USD 1,234" or "USD1234"
```

```
# 2. "MN 1,234" (pesos nacionales label)
```

```
# 3. "$1,234" (generic peso sign, assumed MN)
```

```
# Returns (None, None) if none of the patterns match,
```

```
# so the caller can skip the listing safely.
```

```
def parse_price_currency(text):
```

```
    m = re.search(r'USD\s*([\d,]+(?:\.\d+)?)', text, re.IGNORECASE) # -- pattern 1: explicit  
USD label
```

```

if m:
    try:
        return float(m.group(1).replace(",", "")), "USD"
    except Exception:
        pass
    m = re.search(r'MN\s*([\d,]+(?:\.\d+)?)', text, re.IGNORECASE) # -- pattern 2: explicit
    MN (pesos) label
    if m:
        try:
            return float(m.group(1).replace(",", "")), "MN"
        except Exception:
            pass
    m = re.search(r'\$\s*([\d,]+(?:\.\d+)?)', text) # -- pattern 3: generic $ sign; assumed
    MXN by default
    if m:
        try:
            return float(m.group(1).replace(",", "")), "MN"
        except Exception:
            pass
    return None, None # -- unparseable price; caller should skip this listing

```

parse_neighborhood — Extracts neighborhood from a listing card

The location element may appear as a div or span with
a class containing "location". We strip the city and
state suffixes (e.g. ", Baja California Norte") because
Vivanuncios appends them inconsistently, and they
would fragment the neighborhood groupings in the dashboard.

def parse_neighborhood(card, city_name):

loc_el = card.select_one("div[class*='location']") or
card.select_one("span[class*='location']")

if loc_el:

text = loc_el.get_text(strip=True)

for suffix in [f", {city_name}", f", {city_name}", ", Baja California Norte",
", Ciudad de México", ", Nuevo León", ", Jalisco"]:

text = text.replace(suffix, "") # -- remove city/state suffixes appended by

Vivanuncios

return text.strip() or city_name

return city_name # -- fallback to city name when no location element is found

extract_city — Paginates through listings for one city

Opens a persistent requests.Session to reuse TCP

connections and carry cookies across pages. We warm up

the session with a homepage request first so we arrive

on listing pages with a valid cookie jar.

def extract_city(city_name, url_template, target=300):

records = []

```

    session = requests.Session() # -- reuse TCP connections and carry cookies across
pages

    try:
        session.get("https://www.vivanuncios.com.mx/", headers=HEADERS, timeout=10) #
-- warm up session with homepage cookie
        time.sleep(random.uniform(1.0, 2.0))
    except Exception:
        pass

    page = 1
    while len(records) < target:
        url = url_template.format(page=page)
        print(f" [{city_name.upper()}] Page {page}")

        try:
            res = session.get(url, headers=HEADERS, timeout=15)

            if res.status_code == 404:
                print(f" Page {page} not found — stopping")
                break
            if res.status_code != 200:
                print(f" HTTP {res.status_code} — stopping")
                break

            soup = BeautifulSoup(res.text, "html.parser")
            cards = soup.select("div.postingsList-module__card-container") # -- CSS class
for each listing card

            if not cards:
                print(f" No listings on page {page} — stopping")
                break

            for card in cards:
                try:
                    price_el = card.select_one("[data-qa='POSTING_CARD_PRICE']") # --
data-qa attribute targets the price element reliably
                    if not price_el:
                        continue
                    price_text = price_el.get_text(strip=True)
                    price, currency = parse_price_currency(price_text)
                    if price is None:
                        continue # -- skip listings with unparseable prices

                    neighborhood = parse_neighborhood(card, city_name)

                    records.append({
                        "city": city_name,

```

```

        "neighborhood": neighborhood,
        "price": price,
        "currency": currency,
        "scraped_date": date.today().strftime("%Y-%m-%d"), # -- timestamp each
record with today's date
    })
except Exception:
    continue -- skip malformed individual cards without crashing the whole page

print(f" {len(cards)} listings — Total: {len(records)}")

if len(records) >= target:
    print(f" Target {target} reached!")
    break

page += 1
time.sleep(random.uniform(1.5, 3.0)) # -- random delay between pages to mimic
human browsing speed

```

```

except Exception as e:
    print(f" Error on page {page}: {e}")
    time.sleep(3)
    page += 1
    continue

```

```

return records

```

```

# transform — Splits records by currency and applies price filters
# We separate USD and MN records into two lists because
# the dashboard loads them from separate CSVs and applies
# the exchange rate conversion only to USD listings.
# Price thresholds remove obvious outliers (e.g. $1 test
# listings or $5M data entry errors) that would skew medians.
def transform(records):
    usd = [r for r in records if r["currency"] == "USD" and 50 <= r["price"] <= 50000] # --
filter implausible USD prices
    mn = [r for r in records if r["currency"] == "MN" and 1000 <= r["price"] <= 500000] # --
filter implausible MXN prices
    return usd, mn

```

```

# save_csv — Writes one city+currency slice to its own CSV
# Each city gets two files: {city}_mn.csv and {city}_usd.csv.
# Keeping them separate lets load_rentals() in dashboard.py
# apply currency conversion selectively without re-filtering
# a combined file on every page load.
def save_csv(records, city_name, currency):

```

```

filename = f"{city_name.lower()}_{currency.lower()}.csv" # -- e.g. tijuana_mn.csv
path = os.path.join(DATA_DIR, filename)
with open(path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=CSV_FIELDS)
    writer.writeheader()
    for r in records:
        writer.writerow({k: r.get(k, "") for k in CSV_FIELDS})
print(f" [CSV] Saved: {filename} ({len(records)} records)")

# save_mysql — Persists scraped records through stored procedures
# Each listing is saved via two stored procedures:
#   sp_insert_scraped_data — logs the raw URL + title in a
#   scraping audit table for traceability.
#   sp_insert_rental — inserts the cleaned listing into the
#   rentals table with its session_id for grouping.
# Using stored procedures keeps SQL logic in MySQL and
# lets the database enforce constraints (unique indexes, etc.).
def save_mysql(records, city_name, session_id, conn, cursor):
    url = CITIES.get(city_name, "vivanuncios.com.mx").format(page=1) # -- source URL
    for the audit log
    for r in records:
        title = (
            f"City: {r['city']} | Neighborhood: {r['neighborhood']} | "
            f"Price: {r['price']} {r['currency']} | Date: {r['scraped_date']}"
        )
        cursor.callproc("sp_insert_scraped_data", [url, title]) # -- audit: log raw scraped
item
        cursor.callproc("sp_insert_rental", [
            r["neighborhood"], r["city"],
            float(r["price"]), r["currency"],
            0.0, session_id, r["scraped_date"], # -- 0.0 placeholder for square
meters (not available on Vivanuncios)
        ])
        conn.commit()
    print(f" [MySQL] {len(records)} records saved for {city_name}")

# scrape_vivanuncios — ETL Orchestrator (called by main.py)
# Iterates over all cities, runs Extract → Transform → Load,
# and wraps the whole run in an ETL audit log via MySQL
# stored procedures (sp_log_etl_start / sp_log_etl_finish).
# Old CSV files are deleted before each run to prevent
# stale data from mixing with new scrape results.
# Returns the total listing count for the Streamlit UI.
def scrape_vivanuncios(pages=10):
    for city in CITIES:
        for currency in ["usd", "mn"]:

```

```

    path = os.path.join(DATA_DIR, f'{city.lower()}_{currency}.csv')
    if os.path.exists(path):
        os.remove(path) # -- delete stale CSV before re-scraping to avoid mixed data

    conn = get_connection()
    cursor = conn.cursor()

    cursor.callproc("sp_log_etl_start", ["scraping_etl", "Vivanuncios"]) # -- open ETL audit
log entry
    conn.commit()
    log_id = 0
    for result in cursor.stored_results():
        log_id = result.fetchone()[0] # -- capture log_id for the finish call

    cursor.callproc("sp_clear_scraped") # -- delete previous scraping run in MySQL (full
refresh)
    conn.commit()

    total = 0

    try:
        for city_name, url_template in CITIES.items():
            print(f"\n[SCRAPER] Starting: {city_name.upper()}")

            cursor.callproc("sp_start_session", [city_name]) # -- opens a scraping session
row per city
            conn.commit()
            session_id = 0
            for result in cursor.stored_results():
                session_id = result.fetchone()[0] # -- session_id groups all listings from this city
run
            raw = extract_city(city_name, url_template, target=300) # -- scrape up to 300
listings
            usd, mn = transform(raw) # -- split by currency and filter
outliers

            if usd:
                save_csv(usd, city_name, "usd") # -- write USD slice to {city}_usd.csv
            if mn:
                save_csv(mn, city_name, "mn") # -- write MXN slice to {city}_mn.csv

            if raw:
                save_mysql(raw, city_name, session_id, conn, cursor) # -- persist all records
to MySQL

            cursor.callproc("sp_close_session", [session_id, pages, len(raw), "success"]) # --
mark session complete

```



```

        conn.commit()

        total += len(raw)
        print(f" Total {city_name}: {len(raw)} listings")

    except Exception as e:
        print(f"[SCRAPER] Error: {e}")
        cursor.callproc("sp_log_etl_finish", [log_id, total, "error", str(e)]) # -- log error state if
scraping fails mid-run
        conn.commit()
    finally:
        cursor.callproc("sp_log_etl_finish", [log_id, total, "success", None]) # -- always close
the audit log
        conn.commit()
        cursor.close()
        conn.close()

    print(f"\n[SCRAPER] ETL complete — {total} total records")
    return total # -- returned to Streamlit so it can display "X listings extracted"

if __name__ == "__main__":
    scrape_vivanuncios() # -- allows running this module directly for a manual scrape
trigger

```

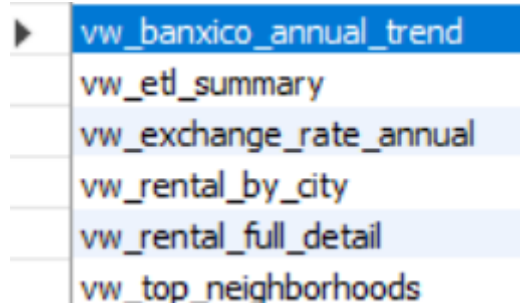
Tablas que tenemos

	Tables_in_cafeteria
▶	audit
	banxicodata
	banxicoserie
	city
	currencysnapshot
	etllog
	inflationindex
	neighborhood
	rentallisting
	scrapeddata

Tabla	Descripción
City	Se analizan y comparan las 4 ciudades usadas (Tijuana, Cd.México, Monterrey, Guadalajara)
Neighborhood	Colonias extraídas del scraping, vinculadas a una ciudad.
ScrapedSession	Registra cada ejecución del scraper: cuándo inició, cuántas páginas procesó, cuántos listados encontró y el estado
ScrapedData	Son los datos crudos del scraping
RentalListing	Datos limpios de cada renta(precio, tipo de cambio, metros cuadrados, etc)
BanxicoSerie	Se catalogaron las series económicas usadas
BanxicoData	Valores de cada serie por fecha. Es la tabla principal de datos económicos con el valor numérico y cuando fue extraído
CurrencySnapshot	Tipo de cambio derivado de los datos de Banxico
InflationIndex	Promedio anual del índice de inflación
ETLLog	Bitácora de las ejecuciones, registra inicio, fin, registros cargados, estado y mensajes de error
Audit	Tabla de auditoría

Views

Hicimos 6 views:

A screenshot of a database interface showing a list of six views. The first view, 'vw_banxico_annual_trend', is highlighted with a blue background. The other views are 'vw_etl_summary', 'vw_exchange_rate_annual', 'vw_rental_by_city', 'vw_rental_full_detail', and 'vw_top_neighborhoods', which are highlighted with a light blue background.

▶	vw_banxico_annual_trend
	vw_etl_summary
	vw_exchange_rate_annual
	vw_rental_by_city
	vw_rental_full_detail
	vw_top_neighborhoods

- **vw_banxico_annual_trend:** Muestra el promedio anual de cada indicador económico(tipo de cambio, inflación, etc)
- **vw_exchange_rate_annual:** Muestra el promedio anual del tipo de cambio
- **vw_rental_by_city:** Resumen de las rentas por ciudad y moneda
- **vw_rental_full_detail:** Vista completa de cada listado de renta por ciudad, colonia, precio, moneda, metros cuadrados y fecha
- **vw_top_neighborhoods:** Muestra las colonia más caras por ciudad

Stored Procedures

sp_insert_banxico	Inserta un valor histórico de Banxico, busca automáticamente “serie_id” por la clave
sp_insert_scraped_data	Guarda los datos crudos del scraping
sp_insert_rental	Inserta una renta, si la colonia no existe la crea automaticamente
sp_start_session	Abre una nueva sesión de scraping
sp_close_session	Cierra sesión registrando paginas, total de enlistados y estado final
sp_log_etl_start	Registra el inicio de un proceso ETL en la bitácora.
sp_log_etl_finish	Cierra el registro ETL con resultado, registros cargados y error si lo hubo.
sp_clear_banxico	Limpia toda la tabla de datos de Banxico para re-sincronizar.
sp_clear_scraped	Limpia los datos de scraping para volver a cargar desde cero.
sp_get_banxico	Devuelve todos los datos de Banxico ordenados por fecha.
sp_get_scraped_data	Devuelve los datos crudos del scraping, del más reciente al más antiguo.
sp_get_rentals	Llama a la vista vw_rental_full_detail para obtener rentas con detalle completo.
sp_insert_serie	Agrega una serie al catálogo. Usa INSERT IGNORE para no duplicar si ya existe.

Triggers

trg_banxico_insert	Cada vez que se inserta un nuevo valor de Banxico, registra automáticamente en la tabla de auditoria
trg_banxico_delete	Si se elimina un dato de Banxico, queda registrado en auditoría con el valor que tenía antes de borrarse
trg_rental_insert	Cada nueva renta insertada genera un registro de auditoría con el precio, moneda y metros cuadrados
trg_rental_delete	Si se borra un listado de renta, la auditoría guarda el precio y el tipo de cambio que tenía
trg_session_update	Cuando una sesión cambia de estado se registra el estado anterior y el nuevo en la tabla de auditoría
trg_etl_update	Cuando el log ETL se actualiza al terminar un proceso, registra el cambio de estado en auditoría.

INSERTS

El código contiene varios INSERT cuya función principal es almacenar información dentro de la base de datos. Permiten registrar ciudades, indicadores económicos de Banxico, datos obtenidos mediante scraping, rentas y los procesos del ETL.

Algunos de los inserts son ejecutados automáticamente mediante los triggers para llevar un control de auditoría sobre las operaciones realizada.

Librerías usadas en Python

- **Streamlit:** Se uso para crear el dashboard
- **Pandas:** Para transformar los datos en tablaas
- **Ploty:** Generar las gráficas del dashboard
- **Request:** Para llamar las API de Banxico y descargar las páginas de Vivanuncios para el scraper
- **Beautifulsoup4:** Se uso para extraer el HTML de la pagina web, usando los datos de renta
- **mysql-connector-python:** Conectar SQL con Python
- **python-dotenv:** Lee el archivo .env para cargar las credenciales

Dashboard

